

Bulletproofs: applications

July 31, 2025

Distributed Lab

 zkdl-camp.github.io

 github.com/ZKDL-Camp

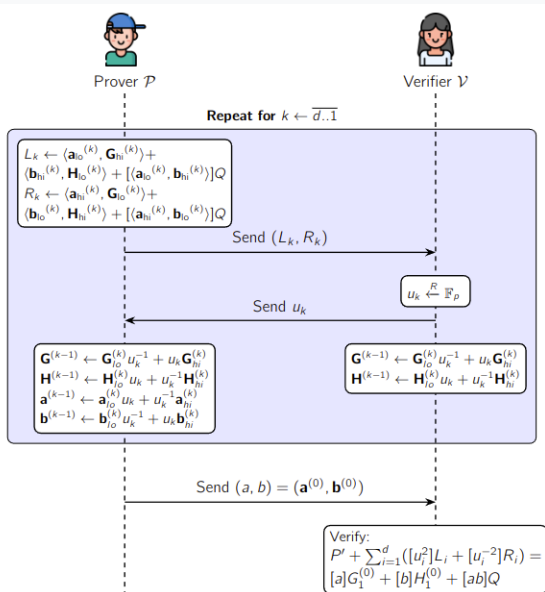


Plan

- 1 Introduction
- 2 IPA polynomial commitment scheme
- 3 Range proofs
- 4 Arithmetic circuits
- 5 Security, efficiency, and open questions

Introduction

Inner-product argument: illustration



Recap: inner-product argument

- **Goal:** Prove $\langle a, b \rangle = c$ with logarithmic proof size
- **Commitment:** $P' = \langle a, G \rangle + \langle b, H \rangle + [\langle a, b \rangle]Q$
- Protocol recursively compresses vectors, sending L_k, R_k at each step
- **Final check:** $P' + \sum ([u_i^2]L_i + [u_i^{-2}]R_i) = [a]G + [b]H + [ab]Q$

Key property

Proof size is $2 \log_2 n$ group elements + 2 scalars. Protocol is *knowledge sound* and *perfect complete* but not *zero-knowledge*.

Idea

We could provide *zero-knowledge* directly to *inner-product argument* construction or use **zk-mul** protocol for outer construction.

IPA polynomial commitment scheme

Recap: Polynomial commitments

- Polynomial commitment scheme

$$\mathcal{C} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

- **Applications:** SNARKs compiled with IOP + polynomial commitment scheme framework (e.g., Halo, Nova, Spartan, Plonk)
- **Desirable properties:** sublinear size, efficient, no trusted setup

Example

One famous example is the **KZG** polynomial commitment scheme, which uses bilinear pairings and requires a trusted setup.

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $\mathcal{C}_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- **Setup** returns independent generators $G = (G_1, \dots, G_n)$.
- **Commit** returns $\text{Com}(f) = \langle f, G \rangle$ where $f = (a_0, \dots, a_{n-1})$
- **Open** given evaluation point $u \in \mathbb{F}_p$ computes $u^n = (1, u, u^2, \dots, u^{n-1})$, obtains $f(u) = \langle f, u^n \rangle$ and runs *inner-product argument* Π_{ip} non-interactively setting

$$a = f, b = u^n, P = \text{Com}(f), c = f(u)$$

to produce an evaluation proof π_{ip}

- **VerifyOpen** given evaluation point $u \in \mathbb{F}_p$ and commitment $\text{Com}(f)$ validates proof π_{ip} running the non-interactive verifier $\mathcal{V}$ of *inner-product argument*.

Range proofs

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v :

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) \mid V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

- **Applications:** Confidential transactions (e.g., Monero, Mumblewimble), other privacy-preserving protocols.
- **Idea:** Prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $\forall i \in 0..n-1 : v_i \in \{0, 1\}$

Naive approach

One could prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $\forall i \in 0..n-1 : v_i \in \{0, 1\}$ using Σ -protocols, but this would be inefficient due to linear proof.

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $a_L = (v_0, v_1, \dots, v_{n-1})$ be the vector of bits padded with zeroes to length n , define $a_R = a_L - 1^n$ so the range validation that v lays in $[0, 2^n)$ implies two checks:

- The following inner-product equality holds: $\langle a_L, 2^n \rangle = v$
- Each bit v_i must be either 0 or 1:

$$a_L - a_R - 1^n = 0^n$$

$$a_L \circ a_R = 0^n$$

Example

Let $a_L = (1, 0, 1, 0)$, $a_R = (0, -1, 0, -1)$, then $a_L \circ a_R = (0, 0, 0, 0)$

Compiling range proof into inner-product

This two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle a_L \circ a_R, y^n \rangle = 0 \text{ and } \langle a_L - a_R - 1^n, y^n \rangle = 0$$

This checks are sound because the prover doesn't know challenge y in advance. So we must combine three inner-product checks:

1. $\langle a_L, 2^n \rangle = v$
2. $\langle a_L, a_R \circ y^n \rangle = 0$
3. $\langle a_L - a_R - 1^n, y^n \rangle = 0$

into one soundly summing up with powers of other challenge $z \in \mathbb{F}_p$:

$$z^2 \cdot \langle a_L, 2^n \rangle + z \cdot \langle a_L - a_R - 1^n, y^n \rangle + \langle a_L, a_R \circ y^n \rangle = z^2 v$$

Compiling range proof into inner-product

Using some dark linear algebra wizardry we could combine the three inner-product checks into a single one inner-product check:

$$\langle a_L - z \cdot 1^n, z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n \rangle = z^2 v + \delta(y, z)$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = (z - z^2) \langle 1^n, y^n \rangle - z^3 \langle 1^n, 2^n \rangle$$

Now it's time to bring out **zk-mul** for inner-products!

Firstly, construct the blinding polynomials for a_L and a_R :

$$a'_L \leftarrow a_L + s_L x \quad a'_R \leftarrow a_R + s_R x$$

Compute polynomials $l(x) = l_0 + l_1 x$, $r(x) = r_0 + r_1 x$:

$$l(x) = a'_L - z \cdot 1^n = (a_L + s_L x) - z \cdot 1^n = a_L - z \cdot 1^n + s_L x$$

$$\begin{aligned} r(x) &= z^2 \cdot 2^n + z \cdot y^n + a'_R \circ y^n = z^2 \cdot 2^n + z \cdot y^n + (a_R + s_R x) \circ y^n \\ &= z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n + s_R \circ y^n x \end{aligned}$$

Bulletproofs range proof: Example

- $v = 19, n = 5, a_L = (1, 1, 0, 0, 1)$
- $v = 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 0 + 8 \cdot 0 + 16 \cdot 1 = 19$
- Prove $a_L \in \{0, 1\}^5$ and $v = \langle a_L, (1, 2, 4, 8, 16) \rangle$
- All done in zero-knowledge

Bulletproofs range proof: Performance

- Proof size: $2 \log_2 n + 9$ group elements (for n -bit range)
- No trusted setup
- Batch proofs: multiple values in one proof
- Efficient for client-side proving

Arithmetic circuits

Proving arithmetic circuit satisfiability

- **Goal:** Prove knowledge of w s.t. $C(w) = 0$ for circuit C
- Encode as R1CS: $Aw \circ Bw = Cw$
- Each constraint is a multiplication of linear forms

Bulletproofs for circuits: Approach

- Prover commits to witness w
- Encodes all constraints as inner-products
- Uses IPA protocol to prove all constraints in zero-knowledge

Batching

All constraints can be aggregated into a single inner-product argument

Bulletproofs for circuits: Example

- Circuit: $w_1 \cdot w_2 = w_3$, $w_3 + w_1 = 7$
- Encode as R1CS, commit to w
- Prove in zero-knowledge using bulletproofs

Bulletproofs for circuits: Performance

- Proof size: $2 \log_2 n + O(1)$ group elements (n = number of constraints)
- No trusted setup
- Efficient for small/medium circuits
- Verification time linear in n

Security, efficiency, and open questions

Security and efficiency

- **Security:** Discrete log assumption, no trusted setup
- **Efficiency:** Logarithmic proof size, fast prover for small n
- **Limitations:** Verification time linear in n

Open questions and extensions

- Can we get sublinear verification?
- Can we combine with SNARKs for better efficiency?
- Applications: privacy, blockchains, verifiable computation
- Extensions: batch proofs, recursive proofs, Halo/Nova

Summary

- Bulletproofs: efficient, no trusted setup, versatile
- IPA polynomial commitment: core primitive
- Range proofs and circuit proofs: practical applications
- Still active area of research!

Thank you!

Questions?